

# Architectural Justification

## *Disaster Relief Supply & Volunteer Coordinator*

**Architecture Selection:** “We chose a Microservices architecture for the Disaster Relief Supply & Volunteer Coordinator system.”

### **Reason 1 — Fault Isolation Across Disconnected Relief Sites**

A disaster zone spans multiple shelters and warehouses that frequently lose connectivity independently of one another. Because the system is decomposed into independently deployable services (Requisition & Priority, Warehouse Inventory, Volunteer Coordination, Logistics Tracking, Notification & Alert), a failure or network partition affecting one service — for example, a warehouse losing its connection — does not bring down volunteer dispatch or vehicle tracking elsewhere. A monolithic or tightly layered design would risk a single failure cascading through the entire platform at the worst possible time.

### **Reason 2 — Independent Scaling Under Surge Load**

Demand across services is highly uneven and unpredictable during an active incident: volunteer registrations can spike in the first hours, while requisition traffic surges as shelters report needs. Microservices let each service scale independently (e.g., the Volunteer Coordination Service can be scaled out during a mass sign-up wave) without over-provisioning the entire system, which matters when compute resources at a disaster response site are limited.

### **Security Advantage**

Each service owns a narrow, well-defined interface (e.g., `IInventoryQuery`, `IVolunteerDispatch`) and can be independently authenticated and network-segmented. If one service is compromised — for instance, the public-facing Disaster Mapping Dashboard — the blast radius is limited to what that service's required interfaces expose, rather than granting an attacker direct access to the Warehouse Inventory or Volunteer Coordination data stores.

### **Performance Benefit**

The Disaster Mapping Dashboard can cache responses from the Warehouse Inventory and Logistics Tracking services locally (satisfying the offline GIS caching requirement) without being blocked by, or blocking, the Requisition & Priority Service's real-time low-stock alerting pipeline. Decoupled services mean the low-bandwidth, latency-sensitive field dashboard is never held up by heavier backend processing happening elsewhere in the system.

DISASTER RELIEF SUPPLY AND VOLUNTEER COORDINATOR -UML COMPONENT DIAGRAM  
Architecture Style : Microservices

